

Databricks Certified Generative AI Associate

100-Question Practice Set — Set 2

Blueprint-weighted • MCQ + open Q&A • with answer explanations

Section (weight)	Qs
1 · Design Applications (14%)	14
2 · Data Preparation (14%)	14
3 · Application Development (30%)	30
4 · Assembling & Deploying Apps (22%)	22
5 · Governance, Security & Privacy (8%)	8
6 · Evaluation & Monitoring (12%)	12
TOTAL	100

Exam: 45 scored MCQs · 90 minutes · \$200 · 6 weighted sections

Set 2 · all-new questions · no overlap with Set 1

How To Use This Set

These 100 questions are distributed to mirror the official 2026 exam weighting, so the number of questions per section reflects how heavily each area is tested. Application Development (30%) and Assembling & Deploying (22%) carry the most weight — prioritise them.

Each item is either a **multiple-choice question (MCQ)** with options A–D and a marked correct answer, or an **open question** that prompts you to explain a concept in your own words before reading the model answer. Every item includes a short explanation covering *why the answer is right and why the distractors are wrong*.

Newer 2026-tested topics are woven throughout: **Agent Bricks, MCP (Model Context Protocol), AI Gateway (inference/usage tables, rate limiting), MLflow 3 Tracing, Prompt Registry/versioning, Mosaic AI Agent Evaluation, custom scorers, and Lakehouse Monitoring.**

Suggested approach: cover the answer, attempt the question, then check. These 100 are all new and do not repeat Set 1, so working both sets gives you 200 unique questions. Re-test any section where you score below ~80%, and ask for a timed mock exam when you're ready.

Section 1 — Design Applications (14%)

Q1. MCQ

A product team wants an LLM feature shipped in 2 weeks for a well-scoped Q&A use case over internal docs. Which design choice minimizes time-to-value?

- A. Pre-train a foundation model from scratch
- B. Fine-tune a 70B model on all internal docs
- C. Use RAG with a hosted Foundation Model API
- D. Build a custom multi-agent framework

✓ **Answer: C**

RAG over a hosted model needs no training and grounds answers in current docs — fastest path. Pre-training and large fine-tunes are slow/expensive; a custom multi-agent build is over-engineering for scoped Q&A.

Q2. OPEN

What are the four parts of the four-part prompt structure, and which two are most often weak in practice?

🔗 **Model answer:**

Instruction (what to do), Context (background/persona/domain), Input (the data/question), and Output Format (structure/length/style). In practice the Output Format and a clearly-separated Instruction are most often underspecified, which is why outputs come back inconsistent.

Q3. MCQ

You must extract entities from text AND generate a fluent summary in one product. Which is the soundest design?

- A. Force one encoder-only model to do both
- B. Use an encoder-only/classifier stage for extraction and a decoder LLM stage for summarization
- C. Use a diffusion model
- D. Do everything with regex

✓ **Answer: B**

Match each task to the right architecture: extraction/classification suits encoder-style models; fluent generation needs a decoder LLM. One encoder-only model can't generate; diffusion is for images; regex can't summarize.

Q4. MCQ

Which signal indicates you should add a Planning step rather than relying on a single prompt?

- A. The task is a one-line factual lookup
- B. The task has interdependent sub-steps whose order and outputs matter
- C. You only need lower latency
- D. You want a shorter prompt

✓ **Answer: B**

Planning is justified when a task decomposes into ordered, interdependent sub-steps. Single lookups don't need it; planning typically adds latency and length, not reduces them.

Q5. OPEN

When designing prompt/response pairs for a given model task, what does it mean for a pair to 'align' with the task?

🔗 **Model answer:**

Alignment means the prompt's intent and the expected response type match the model's capability and the task: a classification task should pair a classify-instruction with a label response (not free prose); a generation task pairs an open instruction with generated text. Misaligned pairs (e.g., asking an encoder model to generate) produce poor or impossible results.

Q6. MCQ

A bank chatbot must never give investment advice. The cleanest place to encode this constraint at design time is:

- A. Only in the UI tooltip
- B. In the system prompt/guardrails plus an output content filter
- C. By lowering temperature
- D. By increasing k in retrieval

✓ **Answer: B**

Behavioral constraints belong in the system prompt and enforced guardrails/filters. UI text doesn't bind the model; temperature and k don't encode policy.

Q7. MCQ

Which is the BEST candidate task for an Agent Bricks Knowledge Assistant?

- A. Converting PDFs into a fixed JSON schema
- B. Answering employee questions grounded in a governed document corpus
- C. Orchestrating five worker agents in parallel
- D. Training an embedding model

✓ **Answer: B**

The Knowledge Assistant Brick is a managed RAG Q&A assistant over a governed corpus. Structured extraction is the Information Extraction Brick; parallel orchestration is the Supervisor; training embeddings is unrelated.

Q8. OPEN

Why might a single-agent design be preferable to multi-agent even when a task seems complex?

✍ **Model answer:**

Multi-agent adds orchestration overhead, more LLM calls (cost/latency), harder debugging, and more failure surface. If the task can be solved with one agent using tools and good prompting, a single agent is simpler, cheaper, and easier to evaluate. Reach for multi-agent only when subtasks are genuinely independent/parallel or need distinct specialization.

Q9. MCQ

A use case needs strict, schema-valid JSON every time for downstream systems. Which combined design is most robust?

- A. High temperature + vague instruction
- B. Explicit output schema in the prompt + an output parser that validates and re-asks on failure
- C. Few-shot only, no validation
- D. Rely on the model's defaults

✓ **Answer: B**

Specifying the schema and validating with a parser that retries on parse failure is the robust pattern. High temperature hurts; examples alone don't guarantee validity; defaults are unreliable.

Q10. MCQ

You are choosing between fine-tuning and RAG for keeping answers up to date with daily-changing data. Which statement is correct?

- A. Fine-tuning is better for frequently changing facts
- B. RAG is better for frequently changing facts; fine-tuning is better for style/behavior/format adaptation
- C. They are interchangeable
- D. RAG changes model weights

✓ **Answer: B**

RAG injects current data at query time — ideal for volatile facts. Fine-tuning bakes in style, tone, format, or task behavior but is poor for fast-changing facts and does not run at query time. RAG does not modify weights.

Q11. OPEN

Describe the fact-store (knowledge-triple) memory pattern and when it beats full-history injection.

🔗 **Model answer:**

Fact-store memory extracts and stores salient facts (often subject–predicate–object triples) and retrieves only relevant ones into the prompt. It beats full-history injection when conversations are long: it keeps token cost low and relevance high by avoiding dumping the entire transcript, while preserving the durable facts that matter.

Q12. MCQ

A multi-agent insurance-claim system has agents for intake, fraud-check, and payout. What is the supervisor's role?

- A. To replace all worker agents
- B. To route subtasks to the right worker, coordinate order/parallelism, and assemble results
- C. To embed documents
- D. To serve the endpoint

✓ **Answer: B**

The supervisor orchestrates: deciding which worker handles what, managing dependencies/parallelism, and combining outputs. It doesn't replace workers, embed, or serve.

Q13. MCQ

For an image-captioning feature (image in, text out), which model family is appropriate?

- A. Encoder-only text model (BERT)
- B. A multimodal/vision-language model
- C. A tabular gradient-boosted model
- D. A pure embedding model

✓ **Answer: B**

Image-to-text requires a multimodal vision-language model. BERT is text-only; gradient boosting is tabular; an embedding model produces vectors, not captions.

Q14. OPEN

What is prompt injection, and why is it primarily a design-time concern rather than only a runtime one?

🔗 **Model answer:**

Prompt injection is when untrusted input contains instructions that hijack the model's behavior (e.g., 'ignore previous instructions'). It's a design-time concern because mitigations must be architected in: delimiting/escaping untrusted input, separating system vs user roles, validating inputs, restricting tool permissions, and adding guardrails. Bolting it on at runtime alone is insufficient.

Section 2 — Data Preparation (14%)

Q15. MCQ

A document set has long sections with distinct subtopics. Embeddings retrieve whole pages that are only partly relevant. Best fix?

- A. Increase chunk size to whole documents
- B. Use smaller, semantically-bounded chunks (e.g., per subsection) with modest overlap
- C. Remove overlap entirely
- D. Switch the LLM

✓ **Answer: B**

Smaller, topic-bounded chunks sharpen embedding focus so retrieval returns the relevant subsection, not a whole noisy page. Bigger chunks worsen dilution; removing overlap risks boundary loss; the LLM isn't the issue.

Q16. OPEN

Explain why embedding dimensionality and the distance metric must be consistent between index creation and querying.

🔗 **Model answer:**

Vector search compares query and document vectors using a distance/similarity metric (cosine, L2, dot). Both must have the same dimensionality (same model) and the index must be queried with the same metric it was built for; mismatches make distances meaningless or throw errors, destroying retrieval quality.

Q17. MCQ

You need to extract only paragraphs mentioning 'refund' from millions of reviews before chunking. Which approach scales best on Databricks?

- A. Open each file manually
- B. A Spark/SQL filtering job (or ai_query/UDF) over a Delta table
- C. Copy-paste into a notebook cell
- D. Email the dataset to reviewers

✓ **Answer: B**

Distributed filtering with Spark/SQL (optionally ai_query or a UDF) over Delta scales to millions of rows. Manual/clipboard/email approaches don't scale and aren't reproducible.

Q18. MCQ

Which is a valid reason to normalize/clean text (lowercasing where appropriate, removing control chars, fixing encoding) before embedding?

- A. It guarantees zero hallucination
- B. It reduces noise so embeddings better capture semantic content and improves match consistency
- C. It changes the embedding model
- D. It is never recommended

✓ **Answer: B**

Cleaning reduces noise (garbled chars, boilerplate) so vectors reflect true content and matching is more consistent. It doesn't eliminate hallucination or alter the model, and it is recommended.

Q19. OPEN

What is the difference between document-level and chunk-level metadata, and give an example use of each in retrieval.

🔗 **Model answer:**

Document-level metadata describes the whole source (author, doc_type, publish_date, access_tier) and is typically copied onto each chunk for filtering (e.g., restrict to doc_type='policy'). Chunk-level metadata describes the piece (section heading, page number, position) and supports citation and reranking (e.g., show 'Section 3, p.12' as the source).

Q20. MCQ

A RAG system over rapidly-updated product specs returns stale answers. The vector index is a Delta Sync index. What is the most likely issue?

- A. Sync is misconfigured or the source Delta table isn't updated/CDF enabled
- B. The LLM temperature is too low
- C. Chunks are too small
- D. Too few few-shot examples

✓ **Answer: A**

Stale results from a Delta Sync index point to a sync/source problem: source table not updated, Change Data Feed not enabled, or sync not triggered. Temperature, chunk size, and examples don't cause staleness.

Q21. MCQ

For semantic retrieval over short tweets vs long contracts, the embedding model's context length matters MOST for:

- A. Tweets
- B. Contracts (long chunks risk truncation)
- C. Neither
- D. Only the query side

✓ **Answer: B**

Long documents create long chunks that can exceed the embedding model's max sequence length and get truncated, losing information. Short tweets rarely hit the limit. So context length matters most for long-document chunks.

Q22. OPEN

Why is it good practice to store a stable chunk_id and source reference with each embedded chunk?

🔗 **Model answer:**

A stable chunk_id and source pointer (doc id, URL, page/section) enable citations and traceability, allow targeted updates/deletes when a source changes, support deduplication, and let you join retrieval results back to governed source data for auditing and access control.

Q23. MCQ

Your corpus has many near-duplicate documents inflating retrieval with redundant chunks. Best data-prep mitigation?

- A. Deduplicate/cluster near-identical content before indexing
- B. Increase temperature
- C. Add more duplicates
- D. Disable metadata

✓ **Answer: A**

Deduplicating near-identical content prevents the retriever from returning many redundant chunks, improving diversity and precision of context. The other options don't address redundancy.

Q24. MCQ

Which preprocessing best supports accurate citations back to the original PDF page?

- A. Strip all positional info
- B. Preserve page numbers and section headings as chunk metadata during extraction
- C. Merge all pages into one blob
- D. Randomize chunk order

✓ **Answer: B**

Keeping page/section info as metadata lets the app cite the exact location. Stripping position, merging pages, or randomizing destroys the ability to point back to the source.

Q25. **OPEN**

Describe how you would prepare a mixed corpus (HTML pages, PDFs, and Markdown) for a single RAG index.

Model answer:

Parse each format with an appropriate loader (e.g., unstructured for PDFs/HTML, a markdown splitter for MD), strip boilerplate, normalize to clean text, apply structure-aware chunking with overlap, attach consistent metadata (source, type, section, ids), land chunks in a Delta table, then embed with one shared embedding model and sync into one Vector Search index.

Q26. **MCQ**

A team sees high retrieval recall but low precision (lots of irrelevant chunks). Which adjustment most directly helps precision?

- A. Add a reranking step and/or tighten metadata filters and lower k
- B. Increase k drastically
- C. Remove all metadata
- D. Increase chunk size to whole docs

Answer: A

Reranking, tighter filters, and a smaller k cut irrelevant results, raising precision. Increasing k, removing metadata, or enlarging chunks generally lowers precision.

Q27. **MCQ**

What is the main purpose of chunk overlap expressed as a percentage of chunk size?

- A. To reduce embedding cost
- B. To preserve context that would otherwise be split across chunk boundaries
- C. To increase hallucination
- D. To shuffle data

Answer: B

Overlap repeats a portion of adjacent text so ideas spanning a boundary remain retrievable in at least one chunk. It slightly increases (not reduces) storage/cost and has nothing to do with shuffling or hallucination.

Q28. **OPEN**

Why might you choose a smaller, faster embedding model even if a larger one scores higher on benchmarks?

Model answer:

Embedding runs on every document at ingestion and every query at runtime, so a smaller/faster model lowers latency and cost at scale, and if chunks are short the quality gap may be negligible for the use case. The decision balances retrieval quality against throughput, cost, and the embedding endpoint's context length.

Section 3 — Application Development (30%)

Q29. MCQ

In Databricks SQL, which function lets you call an LLM/served model directly inside a SQL query for batch scoring?

- A. `ai_query()`
- B. `explode()`
- C. `collect_set()`
- D. `vector_search()`

✓ **Answer: A**

`ai_query()` invokes a Foundation Model API or served endpoint from SQL, ideal for batch scoring over a table. `explode/collect_set` are array functions; there's no SQL function literally named `vector_search` for generation.

Q30. OPEN

What is the difference between the Databricks SDK and the OpenAI SDK when querying a Databricks-served LLM?

🔗 **Model answer:**

Both can call a Databricks model-serving endpoint. The Databricks SDK/Python client targets the serving endpoint natively; Databricks Foundation Model and external-model endpoints are also OpenAI-compatible, so you can use the OpenAI SDK by pointing its `base_url` at the Databricks endpoint and authenticating with a Databricks token. Choice is about ecosystem familiarity and portability — both reach the same endpoint.

Q31. MCQ

You log a LangChain chain with mlflow. To make it servable with correct inputs, you should also provide:

- A. A model signature and input example
- B. A screenshot of the notebook
- C. The GPU temperature
- D. A list of unrelated datasets

✓ **Answer: A**

A signature + input example define the I/O contract for validation and serving. Screenshots, hardware temps, and unrelated datasets don't help deployment.

Q32. MCQ

A chain occasionally returns malformed JSON, breaking downstream parsing. Best development-time mitigation?

- A. Ignore failures
- B. Use a structured output parser with retry/repair (e.g., re-prompt on parse error)
- C. Raise temperature
- D. Remove the schema requirement

✓ **Answer: B**

A parser that validates and re-asks/repairs on failure makes output robust. Ignoring failures breaks downstream; higher temperature worsens it; dropping the schema defeats the purpose.

Q33. OPEN

What does it mean to 'compose' a RAG chain, and what are the minimum components?

🔗 **Model answer:**

Composing means wiring modular components into a pipeline where each step's output feeds the next. Minimum components: a retriever (embeds the query and fetches top-k context from a vector store), a prompt template (assembles instruction + retrieved context + question), an LLM (generates the answer), and usually an output parser (structures/validates the result).

Q34. MCQ

Which is TRUE about provisioned throughput Foundation Model endpoints for development?

- A. They guarantee dedicated capacity/throughput for steady workloads
- B. They are always cheaper than pay-per-token
- C. They cannot be logged with MLflow
- D. They require no endpoint at all

✓ **Answer: A**

Provisioned throughput reserves dedicated capacity for predictable, steady, high-volume workloads. It is not always cheaper (idle cost), models/chains using it are still MLflow-trackable, and it is an endpoint.

Q35. MCQ

To let an agent safely query a governed table as a tool, the recommended Databricks mechanism is:

- A. A Unity Catalog function exposed as a tool
- B. Hard-coded credentials in the prompt
- C. A public CSV link
- D. Disabling permissions

✓ **Answer: A**

Unity Catalog functions can be registered as governed tools the agent calls, keeping permissions and lineage intact. Hard-coded creds, public links, and disabling permissions are insecure.

Q36. OPEN

What is the role of an output parser vs a guardrail in a chain — how do they differ?

🔗 **Model answer:**

An output parser transforms the model's raw text into a structured, validated object (format/schema correctness). A guardrail enforces safety/policy/content constraints (toxicity, PII, topic scope, injection) on inputs and/or outputs. Parsing is about shape; guardrails are about acceptability. A robust app uses both.

Q37. MCQ

You want to compare two prompt templates' quality on a fixed dataset before shipping. Best tool?

- A. `mlflow.evaluate()` over both variants with the same eval set
- B. Eyeballing 2 examples
- C. Changing the embedding model
- D. Increasing k

✓ **Answer: A**

Running `mlflow.evaluate()` on both prompt variants against the same dataset gives comparable, quantitative metrics. Eyeballing is unreliable; embeddings/k aren't the variable under test.

Q38. MCQ

Which statement about temperature and top_p is correct?

- A. Both control output randomness; tuning one is usually enough, and lower values make output more deterministic
- B. They control retrieval k
- C. They set the embedding dimension
- D. They are governance settings

✓ **Answer: A**

Temperature and top_p (nucleus sampling) both shape output randomness; commonly you tune one. Lower values yield more deterministic output. They are decoding params, not retrieval/embedding/governance settings.

Q39. OPEN

Explain how function/tool calling works end to end in an LLM application.

🔗 **Model answer:**

You provide the model tool definitions (name, description, JSON parameter schema). Given a query, the model decides whether to call a tool and emits a structured call with arguments. Your application executes the real function, returns the result to the model, and the model incorporates it to produce the final answer — optionally calling more tools in a loop.

Q40. MCQ

A streaming chat UI needs tokens as they are produced. Which serving capability supports this?

- A. Token streaming from the model-serving endpoint
- B. Batch inference
- C. Delta time travel
- D. Photon

✓ **Answer: A**

Streaming endpoints emit tokens incrementally for responsive UIs. Batch inference returns whole results offline; time travel and Photon are unrelated to token streaming.

Q41. MCQ

You need an agent to remember a user's name across a long session without re-sending the whole transcript. Best approach?

- A. Full-history injection every turn
- B. Extract key facts into a small memory store and inject only those
- C. Fine-tune per user
- D. Increase max tokens to infinity

✓ **Answer: B**

A fact/memory store injecting only salient facts keeps prompts short while preserving needed info. Full-history is token-heavy; per-user fine-tuning is impractical; you cannot set infinite context.

Q42. OPEN

What problem does the MLflow Prompt Registry solve during iterative development, and how do aliases help?

🔗 **Model answer:**

It version-controls prompts so changes are tracked, comparable, and reversible — preventing untracked prompt drift across notebooks/apps. Aliases (e.g., 'production', 'staging') point to a specific version so code references a stable alias while you swap the underlying version, enabling safe promotion, rollback, and A/B testing without editing application code.

Q43. MCQ

A developer integrates an external (third-party) model via Databricks. Which endpoint type routes to it under unified governance?

- A. External model endpoint (via AI Gateway/Model Serving)
- B. Delta Live Table
- C. Unity Catalog Volume
- D. A notebook widget

✓ **Answer: A**

External model endpoints let Databricks proxy third-party providers under unified serving/AI Gateway governance (keys, rate limits, logging). DLT/Volumes/widgets don't route model traffic.

Q44. MCQ

To reduce hallucination in a RAG chain, which prompt instruction is MOST effective?

- A. 'Be creative and elaborate.'
- B. 'Answer only using the provided context; if it is not present, say you don't know.'
- C. 'Use as many tokens as possible.'
- D. 'Ignore the context.'

✓ **Answer: B**

Constraining the model to the provided context and licensing 'I don't know' directly curbs fabrication. Creativity, verbosity, or ignoring context all increase hallucination risk.

Q45. OPEN

What is the difference between logging a model with a built-in flavor (e.g., langchain) vs a custom PyFunc?

🔗 **Model answer:**

A built-in flavor knows how to serialize/serve a known framework with minimal code. A custom PyFunc wraps arbitrary Python (your own predict logic, pre/post-processing, multiple calls) when no flavor fits. Use the flavor when it matches your stack; use PyFunc for bespoke chains/agents that need custom orchestration.

Q46. MCQ

An agent must call tools in a fixed dependency order (A then B then C). Which design enforces this most reliably?

- A. Let the LLM freely choose order every time
- B. Encode the sequence in a deterministic chain/workflow rather than relying on the model
- C. Raise temperature
- D. Remove tool descriptions

✓ **Answer: B**

When order is fixed and critical, a deterministic chain/workflow guarantees it. Leaving it to the model is non-deterministic; temperature/removing descriptions reduce reliability.

Q47. MCQ

Which is the correct way to pass retrieved context to the LLM in a RAG prompt?

- A. Embed the context into the model weights at runtime
- B. Insert the retrieved chunks into a context section of the prompt template
- C. Send only the chunk IDs
- D. Put context in the model's filename

✓ **Answer: B**

Retrieved chunk text is injected into the prompt's context section so the model can ground its answer. You can't write to weights at runtime; IDs alone carry no content; filenames are irrelevant.

Q48. OPEN

Why is it important to set max output tokens appropriately for a generation endpoint?

🔗 **Model answer:**

Max output tokens caps generation length: too low truncates answers mid-thought; too high wastes cost/latency and, combined with a long input prompt, can exceed the model's total context window and error out. It also interacts with input length — input + output must fit the context window. Set it to the expected answer size with a safety margin.

Q49. MCQ

A team wants the same chain to work with different models (dev vs prod) without code changes. Best practice?

- A. Hard-code the model name everywhere
- B. Parameterize the model/endpoint via config or an alias and inject at runtime
- C. Duplicate the codebase
- D. Recompile for each model

✓ **Answer: B**

Externalizing the model/endpoint into config or an alias lets you swap models without touching code. Hard-coding and duplication cause drift; recompiling isn't how Python chains work.

Q50. MCQ

Which is a correct reason to add a reranker after vector retrieval in a chain?

- A. To re-order candidate chunks by deeper relevance before passing top results to the LLM
- B. To embed documents
- C. To serve the endpoint
- D. To mask PII

✓ **Answer: A**

A reranker (often a cross-encoder) re-scores retrieved candidates for finer relevance, improving the quality of context sent to the generator. It doesn't embed, serve, or mask.

Q51. OPEN

Describe a windowed memory strategy and one drawback.

✍ **Model answer:**

Windowed memory keeps only the most recent N turns (or N tokens) and injects them into the prompt, discarding older turns. Drawback: information from earlier in the conversation falls out of the window and is forgotten, so long-range references or facts stated early can be lost unless also captured in a separate fact store.

Q52. MCQ

To call a Databricks Foundation Model endpoint that is OpenAI-compatible using the OpenAI Python SDK, you must:

- A. Set `base_url` to the Databricks endpoint and use a Databricks token as the API key
- B. Nothing — it works with no configuration
- C. Convert the model to ONNX
- D. Use FTP

✓ **Answer: A**

Point the OpenAI client's `base_url` at the Databricks serving endpoint and authenticate with a Databricks token. No ONNX conversion or FTP is involved, and config is required.

Q53. MCQ

Which approach best handles a query that needs both database facts and document context?

- A. A tool-using agent that queries the DB (UC function) and a retriever for documents, then synthesizes
- B. A single static prompt
- C. Only keyword search
- D. Lowering temperature

✓ **Answer: A**

Combining a structured DB tool and a document retriever within a tool-using agent covers both sources, then the model synthesizes. A static prompt can't fetch live data; keyword-only or temperature changes don't address the dual-source need.

Q54. OPEN

What is the purpose of an 'input example' when logging an MLflow GenAI model?

🔍 **Model answer:**

An input example is a representative sample request stored with the model. MLflow uses it to infer/validate the signature, to build serving input schemas, to test that the model loads and predicts, and to document the expected request format for consumers — reducing deployment surprises.

Q55. MCQ

A developer notices the agent loops forever retrying a failing tool. Best safeguard?

- A. Add a max-iterations / step limit and graceful fallback
- B. Remove all tools
- C. Increase temperature
- D. Disable logging

✓ **Answer: A**

A max-iteration limit (and fallback response) prevents infinite loops while keeping the agent useful. Removing tools cripples it; temperature/logging changes don't stop loops.

Q56. MCQ

Which is the BEST description of grounding in a GenAI app?

- A. Connecting outputs to authoritative, retrievable evidence (e.g., retrieved context) to improve factuality
- B. Lowering the GPU clock
- C. Compressing the model
- D. Random sampling

✓ **Answer: A**

Grounding ties responses to authoritative evidence (retrieved docs, tool results) so claims are supported and verifiable. The other options are unrelated hardware/model concepts.

Q57. OPEN

Why should developers separate the system prompt (instructions/role) from user-provided input in code?

🔍 **Model answer:**

Separating system instructions from user content (distinct roles/sections, delimiting untrusted input) reduces prompt-injection risk, keeps policy/behavior stable regardless of user text, and makes prompts easier to version and test. Mixing them lets user input override intended behavior.

Q58. **MCQ**

You want to cache responses for identical or near-identical queries to cut cost/latency. Which is the soundest approach?

- A.** A semantic/exact cache keyed on the (normalized) query, with TTL and invalidation on data updates
- B.** Never cache anything
- C.** Cache forever with no invalidation
- D.** Cache the model weights

✓ **Answer: A**

A query cache (exact or embedding-based semantic match) with a TTL and invalidation when source data changes reduces cost/latency while avoiding stale answers. Never caching wastes money on repeats; caching forever serves stale results; weights aren't a response cache.

Section 4 — Assembling & Deploying Applications (22%)

Q59. MCQ

After registering a model to Unity Catalog, what is the next step to expose it for real-time requests?

- A. Create a Model Serving endpoint for that registered model version
- B. Delete the model
- C. Export to CSV
- D. Email the artifact

✓ **Answer: A**

Creating a Model Serving endpoint backed by the registered UC model version exposes it as a scalable REST API. The other options don't serve the model.

Q60. OPEN

List the three-level Unity Catalog namespace for a registered model and why it matters for deployment.

🔗 **Model answer:**

It's `catalog.schema.model` (e.g., `prod.genai.support_rag`). It matters because it gives a globally unique, governed identifier with fine-grained permissions and cross-workspace access, enabling clean promotion across environments (dev/staging/prod catalogs) and reliable references from serving endpoints and pipelines.

Q61. MCQ

You need to capture every production request/response for a served RAG app to monitor and later evaluate. Enable:

- A. Inference tables on the serving endpoint
- B. Photon
- C. Delta Live Tables only
- D. A bigger embedding model

✓ **Answer: A**

Inference tables auto-log endpoint traffic to Delta for monitoring/evaluation. Photon, DLT, and embeddings don't capture serving payloads.

Q62. MCQ

Which is the correct trigger to move from pay-per-token to provisioned throughput in production?

- A. Traffic becomes high and steady, needing predictable latency/throughput
- B. Traffic drops to near zero
- C. You want to stop monitoring
- D. You delete Unity Catalog

✓ **Answer: A**

High, steady volume with latency/throughput guarantees is the case for provisioned throughput. Near-zero traffic favors pay-per-token; monitoring and governance are unrelated to the choice.

Q63. OPEN

Explain how AI Gateway helps enforce cost control and access policy across multiple LLM endpoints.

🔗 **Model answer:**

AI Gateway sits in front of endpoints and applies centralized policies: rate limits (per user/endpoint) to cap spend and abuse, usage tables to track tokens/cost by consumer, inference tables for request/response logging, permission controls for who can call what, plus routing/fallback. This centralizes governance instead of configuring each endpoint separately.

Q64. MCQ

A canary rollout sends 10% of traffic to a new model version. Which serving feature enables this?

- A. Traffic splitting across served entities on one endpoint
- B. Deleting the old version
- C. Time travel
- D. A notebook comment

✓ **Answer: A**

Model Serving supports multiple served entities on an endpoint with configurable traffic percentages, enabling canary/A-B rollouts. Deleting versions, time travel, and comments don't split traffic.

Q65. MCQ

Which combination gives a reproducible, governed promotion from staging to prod?

- A. UC model aliases (e.g., @staging, @prod) + automated eval gates + CI/CD
- B. Manually copying notebooks
- C. Renaming files by hand
- D. Verbal approval only

✓ **Answer: A**

Aliases let code reference a stable pointer while versions change; eval gates ensure quality; CI/CD automates and audits promotion. Manual copying/rename/verbal approval aren't reproducible or governed.

Q66. OPEN

What is a cold start on a scale-to-zero endpoint, and how do teams mitigate its impact?

🔗 **Model answer:**

With scale-to-zero, compute is released when idle, so the first request after idling must spin up resources, adding latency (cold start). Mitigations: keep a minimum of one replica warm for latency-sensitive apps, use provisioned throughput for steady traffic, pre-warm before known peaks, or accept cold starts for cost-sensitive, latency-tolerant workloads.

Q67. MCQ

To serve a chain that needs a specific Python library version, you ensure correct deployment by:

- A. Logging dependencies/conda or pip requirements with the MLflow model
- B. Hoping prod has it
- C. Installing it on your laptop only
- D. Ignoring versions

✓ **Answer: A**

MLflow captures the environment (pip/conda requirements) with the model so serving rebuilds the right dependencies. Relying on prod, your laptop, or ignoring versions causes deployment failures.

Q68. MCQ

Which is the recommended artifact to deploy when your app includes retrieval + prompt + LLM + parsing as one unit?

- A. A single logged MLflow model (chain/PyFunc) representing the whole app
- B. Four separate ungoverned scripts
- C. A spreadsheet
- D. A screenshot

✓ **Answer: A**

Packaging the full app as one MLflow model gives atomic versioning, one endpoint, a unified signature, and tracing. Disconnected scripts are brittle; spreadsheets/screenshots aren't deployable.

Q69. OPEN

Why is automated evaluation part of a GenAI CI/CD pipeline, and what happens if a metric regresses?

🔗 **Model answer:**

Automated eval (e.g., `mlflow.evaluate` with faithfulness/relevance/correctness on a golden set) provides an objective quality gate so changes don't silently degrade the app. If a metric regresses below threshold, the pipeline fails the build/blocks promotion, prompting investigation/rollback before the change reaches users.

Q70. MCQ

A served endpoint must restrict who can call it. Which mechanism applies?

- A. Endpoint permissions via Unity Catalog / access controls (and AI Gateway policies)
- B. Raising temperature
- C. Increasing k
- D. Adding more chunks

✓ **Answer: A**

Access to serving endpoints is governed by permissions/ACLs (and AI Gateway). Decoding params and retrieval settings don't control authorization.

Q71. MCQ

What is the purpose of a health/readiness check on a serving endpoint during deployment?

- A. To verify the model is loaded and ready before routing traffic
- B. To embed documents
- C. To delete logs
- D. To raise temperature

✓ **Answer: A**

Readiness checks confirm the endpoint can serve before traffic is routed, avoiding errors during startup. The other options are unrelated.

Q72. OPEN

How would you roll back a bad model deployment using Unity Catalog and Model Serving?

🔗 **Model answer:**

Because UC keeps prior model versions and serving references a version (often via an alias), you roll back by repointing the @prod alias (or endpoint's served entity) to the last known-good version, or shifting traffic split back to it. No code change is needed; the previous artifact and its environment are preserved for immediate restoration.

Q73. MCQ

For a nightly job that classifies 5M documents, the most cost-effective serving pattern is:

- A. Batch inference (e.g., ai_query/batch scoring) rather than holding a real-time endpoint hot
- B. A real-time endpoint at max concurrency all day
- C. Manual one-by-one calls
- D. Streaming UI

✓ **Answer: A**

Large offline workloads are cheaper via batch scoring/ai_query than keeping a hot real-time endpoint. Always-on max concurrency wastes money; manual calls don't scale; streaming UIs are for interactivity.

Q74. MCQ

Which statement about external model endpoints is correct?

- A. They let you route to third-party providers under Databricks governance with managed keys
- B. They only work offline
- C. They bypass all logging
- D. They cannot be rate limited

✓ **Answer: A**

External model endpoints proxy third-party providers with centralized key management, logging, and rate limiting via AI Gateway. They're not offline-only, don't bypass logging, and can be rate limited.

Q75. OPEN

What are usage tables (via AI Gateway) and how do they differ from inference tables?

✍ **Model answer:**

Usage tables record consumption metrics — tokens, request counts, cost attribution per user/endpoint — for billing/cost governance. Inference tables capture the actual request/response payloads (and metadata) for monitoring, debugging, and evaluation. Usage = aggregate consumption accounting; inference = the detailed traffic content.

Q76. MCQ

A deployed app must comply with data residency. Which platform feature most directly supports controlling where models/data live?

- A. Workspace/region selection and Unity Catalog governance
- B. Temperature
- C. Chunk overlap
- D. Few-shot count

✓ **Answer: A**

Region/workspace placement plus UC governance control where data and models reside and who can access them. Decoding and chunking settings have nothing to do with residency.

Q77. MCQ

Which is the safest way to provide secrets (e.g., an external API key) to a deployed endpoint?

- A. Databricks secret scopes referenced by the endpoint, not hard-coded in code/prompt
- B. Pasting the key in the system prompt
- C. Committing it to Git
- D. Printing it in logs

✓ **Answer: A**

Secret scopes keep credentials out of code/prompts/logs and inject them securely at runtime. Prompts, Git commits, and logs expose secrets and are insecure.

Q78. OPEN

Describe the end-to-end assembly of a RAG app into a single deployable unit on Databricks.

Model answer:

Build the chain (retriever over a Vector Search index + prompt template + LLM endpoint + parser); log it with MLflow capturing signature, input example, and dependencies; register it to Unity Catalog (catalog.schema.model); create a Model Serving endpoint (with scale-to-zero or provisioned throughput as fit); enable inference tables and AI Gateway policies; gate promotion with automated evaluation; and use aliases/traffic splitting for safe rollout and rollback.

Q79. MCQ

You see frequent 429 (rate limit) errors from a downstream provider via an external endpoint. Best fix?

- A. Configure rate limits/retries and request higher quota via AI Gateway
- B. Increase temperature
- C. Delete inference tables
- D. Shrink chunks

✓ Answer: A

429s are rate-limit issues — manage them with gateway rate limiting, backoff/retries, and higher quotas. Decoding params, deleting logs, or chunk size don't address provider throttling.

Q80. MCQ

Which best describes the relationship between MLflow and Model Serving in deployment?

- A. MLflow packages/tracks/registers the model; Model Serving hosts the registered model as an endpoint
- B. They are the same thing
- C. Model Serving trains the model
- D. MLflow replaces Unity Catalog

✓ Answer: A

MLflow handles packaging, tracking, and registration; Model Serving hosts the registered artifact as a scalable endpoint. They're distinct; serving doesn't train; MLflow doesn't replace UC governance.

Section 5 — Governance, Security & Privacy (8%)

Q81. MCQ

Which Unity Catalog feature shows the full data-to-model-to-endpoint dependency chain for auditing?

- A. Lineage
- B. Temperature
- C. Token streaming
- D. Chunk overlap

✓ **Answer: A**

UC lineage traces how data flows into features, models, and endpoints — essential for audit and impact analysis. The other options are unrelated runtime/data settings.

Q82. OPEN

What is the difference between detecting PII and masking PII, and where should each occur in a RAG pipeline?

🔗 **Model answer:**

Detection identifies sensitive entities (names, SSNs, cards) in text; masking transforms/redacts them (e.g., '****1234' or '[REDACTED]'). Detection+masking should occur during ingestion before embedding/storing, and again on logged prompts/responses (inference tables), so PII never persists in indexes or logs. Retrieval-time row/column masking via UC adds defense in depth.

Q83. MCQ

A model is governed in Unity Catalog. To let only the support team query its endpoint, you:

- A. Grant endpoint/model permissions to the support group
- B. Raise temperature for them
- C. Give everyone admin
- D. Disable lineage

✓ **Answer: A**

Use permission grants scoped to the support group on the model/endpoint. Temperature is irrelevant; blanket admin violates least privilege; disabling lineage harms auditing.

Q84. MCQ

Which is a legitimate governance reason to prefer an open model with a permissive license?

- A. It allows commercial use, redistribution, and fine-tuning within terms
- B. It guarantees zero hallucination
- C. It removes the need for evaluation
- D. It increases token limits

✓ **Answer: A**

A permissive license clarifies legal rights for commercial use, redistribution, and fine-tuning. Licensing says nothing about hallucination, evaluation needs, or token limits.

Q85. OPEN

Why should access control be enforced at the retrieval/data layer rather than only in the prompt?

🔗 **Model answer:**

Prompt-level instructions ('don't reveal HR data') are advisory and can be bypassed via injection or model error. Enforcing at the data layer — UC permissions, row filters, column masks, metadata ACL filtering at query time — guarantees unauthorized records are never retrieved or seen, providing real, auditable security independent of model behavior.

Q86. MCQ

A compliance team needs to delete a specific customer's data from the RAG system on request (right to erasure). What enables this cleanly?

- A. Stable source/chunk IDs + Delta deletes that propagate to the synced vector index
- B. Raising temperature
- C. A bigger LLM
- D. More few-shot examples

✓ **Answer: A**

Tracking source/chunk IDs lets you delete the right rows in the Delta source; a Delta Sync index then removes the corresponding vectors. Decoding, model size, and examples can't perform targeted erasure.

Q87. MCQ

Which is the BEST practice for handling secrets used by a governed GenAI app?

- A. Store in Databricks secret scopes with scoped access; never in code, prompts, or notebooks
- B. Hard-code in the chain
- C. Share via chat
- D. Print to stdout for debugging

✓ **Answer: A**

Secret scopes with least-privilege access keep credentials secure and auditable. Hard-coding, sharing, or printing secrets exposes them.

Q88. OPEN

How do guardrails complement Unity Catalog governance — what does each cover that the other does not?

🔗 **Model answer:**

Unity Catalog governs data/model access: who can read which rows/columns, run which functions, and call which endpoints, with lineage. Guardrails govern model behavior/content: blocking toxic, off-topic, or PII-leaking outputs and resisting prompt injection at the inference boundary. UC stops unauthorized data reaching the model; guardrails stop unsafe content reaching the user. Together they cover access and behavior.

Section 6 — Evaluation & Monitoring (12%)

Q89. MCQ

Which metric best captures whether a generated answer actually addresses the user's question (regardless of context)?

- A. Answer Relevancy
- B. Context Recall
- C. QPS
- D. Perplexity

✓ **Answer: A**

Answer Relevancy judges whether the response is pertinent to the asked question. Context Recall is a retrieval metric; QPS is throughput; perplexity measures LM fit, not relevance.

Q90. OPEN

In RAG, you have high faithfulness but low answer relevancy. What does this combination suggest?

🔗 **Model answer:**

The answer is well-grounded in the retrieved context (not hallucinating) but doesn't actually answer the user's question — often because retrieval pulled the wrong context, or the question was misinterpreted. Fix the retrieval (query rewriting, better chunks/metadata, reranking) or the prompt's task framing, not the grounding.

Q91. MCQ

You want to evaluate a deployed RAG app on real production traffic offline. Which source feeds the eval?

- A. Inference tables (logged requests/responses)
- B. The cover page of the docs
- C. Random text
- D. The embedding dimension

✓ **Answer: A**

Inference tables capture real production inputs/outputs you can score offline (with judges or against labels) and monitor for drift. The other options aren't evaluation data.

Q92. MCQ

Which is a key mitigation for LLM-as-Judge bias?

- A. Use clear rubrics, reference answers, randomized option order, and validate against human labels
- B. Always trust the judge blindly
- C. Use the model being evaluated as its own judge
- D. Raise temperature on the judge

✓ **Answer: A**

Calibrated rubrics, references, de-biasing techniques (e.g., randomizing order), and human validation reduce judge bias/variance. Blind trust, self-judging, and high judge temperature increase unreliability.

Q93. OPEN

What does MLflow Tracing capture for a RAG chain, and name two debugging questions it helps answer.

🔗 **Model answer:**

Tracing records spans for each step — query embedding, retrieval (which chunks, scores), prompt sent, LLM call (tokens, latency), parsing — with inputs/outputs and timing. It helps answer: (1) 'Did retrieval return the right context, or is bad context causing the wrong answer?' and (2) 'Which step is the latency/cost bottleneck?' It also surfaces tool-call errors in agents.

Q94. MCQ

Which production signal most directly indicates degrading user-perceived quality over time?

- A. Trend in user feedback/thumbs and quality metrics on sampled traffic
- B. Disk color
- C. Number of notebooks
- D. Cluster name

✓ **Answer: A**

Tracking feedback signals and periodic quality scoring on sampled production traffic reveals quality drift. The other options are irrelevant.

Q95. MCQ

Which Lakehouse Monitoring monitor type tracks generic time-stamped metrics (not tied to inference logs)?

- A. TimeSeries monitor
- B. InferenceLog monitor
- C. Snapshot monitor
- D. Judge monitor

✓ **Answer: A**

A TimeSeries monitor tracks metrics over a timestamp column for any table. InferenceLog is specialized for model request logs; Snapshot tracks current-state stats; 'Judge monitor' isn't a type.

Q96. OPEN

Why combine automatic metrics (e.g., faithfulness via LLM judge) with human/SME review in evaluation?

🔗 **Model answer:**

Automatic metrics scale to many examples cheaply but can be biased or miss domain nuance; SME review is accurate and captures business correctness but doesn't scale. Combining them — automatic for broad coverage and continuous monitoring, SME for calibration, golden sets, and edge cases — yields evaluation that is both scalable and trustworthy.

Q97. MCQ

For a summarization task, which metric pair is most appropriate?

- A. ROUGE (recall-oriented overlap) plus a faithfulness/factuality check
- B. QPS and latency only
- C. Precision/Recall for classification
- D. Cosine of embeddings only

✓ **Answer: A**

Summarization is commonly scored with ROUGE for content overlap, complemented by a faithfulness/factuality check so the summary doesn't fabricate. Throughput metrics, classification metrics, or raw cosine alone don't capture summary quality.

Q98. MCQ

A custom business rule says answers must include a valid ticket ID. How do you evaluate this at scale?

- A. Write a custom scorer that checks each answer for a valid ticket-ID pattern
- B. Use BLEU
- C. Use perplexity
- D. Eyeball a few

✓ **Answer: A**

A custom scorer encodes the domain rule (regex/validation) and runs over all eval examples for an objective pass rate. BLEU/perplexity don't check business rules; eyeballing doesn't scale.

Q99. OPEN

What is data/concept drift in a deployed GenAI app, and how can monitoring detect it?

🔗 **Model answer:**

Drift is when production inputs (data drift) or the input-output relationship/expectations (concept drift) change over time versus what the app was built/evaluated for — e.g., new question types or shifting source content. Monitoring detects it by tracking input distributions, retrieval relevance, quality-metric trends, and feedback over time (Lakehouse Monitoring on inference tables), alerting when metrics degrade so you can refresh data, prompts, or models.

Q100

MCQ

Which set of production metrics is most relevant for operating a GenAI endpoint (beyond quality)?

- A. QPS, latency, error rate, and cost/token usage
- B. ROUGE, BLEU, faithfulness only
- C. Chunk size and overlap
- D. Embedding dimension

✓ Answer: A

Operational health is tracked via throughput (QPS), latency, error rate, and cost/token usage (AI Gateway usage tables). ROUGE/BLEU/faithfulness are quality metrics; chunking/embedding params are build-time settings.